SMART LOCKER

Tamper Detection System

using TinyML and Edge Impulse

Arduino Nano 33 BLE Sense • SH1106 OLED • Spring Magnetic Sensor

Project Type	TinyML / Security & Tamper Detection
Board	Arduino Nano 33 BLE Sense
Platform	Edge Impulse Studio
Submission	Academic / Office Submission
Status	Final Report

Arduino Nano 33 BLE Sense • Edge Impulse • Tamper Detection

1. Introduction

This project implements a TinyML-based Smart Locker Tamper Detection System using the Arduino Nano 33 BLE Sense board and Edge Impulse Studio. The system performs real-time monitoring and classification of locker conditions using the onboard IMU (accelerometer + gyroscope), onboard PDM microphone, and a magnetic spring sensor connected to a digital GPIO pin.

The trained Edge Impulse model is deployed directly on the Arduino Nano 33 BLE Sense, where the system performs live inference and indicates detected conditions using LEDs, an SH1106 OLED display, and a buzzer. A hybrid approach is used — combining the TinyML model output with deterministic rule-based overrides for robust real-world performance.

> A unique aspect of this project: a pair of headphones was used as a substitute for a dedicated magnetic reed switch + magnet assembly to simulate door open/close transitions during data collection. In a production deployment, a standard magnetic reed switch with a magnet embedded in the locker door would be used instead.

2. Objectives

The main objectives of this project are:

- To collect vibration, sound, and door-state data from a locker under different tamper conditions.
- To train a TinyML classification model using Edge Impulse Studio.
- To deploy the trained model on an Arduino Nano 33 BLE Sense.
- To classify different locker conditions in real time: idle, normal open, forced vibration, and hammer impact.
- To provide visual and audible fault indication via LEDs, OLED display, and buzzer.
- To implement rule-based override logic for improved real-world detection reliability.

3. Components Used

Component	Qty	Purpose
Arduino Nano 33 BLE Sense	1	Main TinyML microcontroller with onboard IMU and microphone
SH1106 OLED Display (128×64)	1	Real-time status display
Magnetic Spring Sensor (Reed Switch)	1	Door open/close detection on D7
Headphones (substitute)	1	Used as magnetic field source during data collection (see Section 6)
LEDs (4×)	4	Visual fault condition indication (idle, normal, vibration, hammer)
220Ω Resistors (4×)	4	Current-limiting for LEDs
Buzzer	1	Audible tamper alert
Breadboard	1	Prototyping platform
Jumper Wires	As per Required	Component connections
USB Cable	1	Power and programming

4. Software and Tools Used

Software / Tool	Purpose
Arduino IDE 2.x	Firmware development and flashing
Edge Impulse Studio	Dataset management, model training and deployment
Edge Impulse CLI (data forwarder)	Real-time serial data acquisition from board
Node.js / npm	CLI tooling for Edge Impulse
macOS / Windows Terminal	CLI tools and board management

5. Block Diagram & Pin Connections

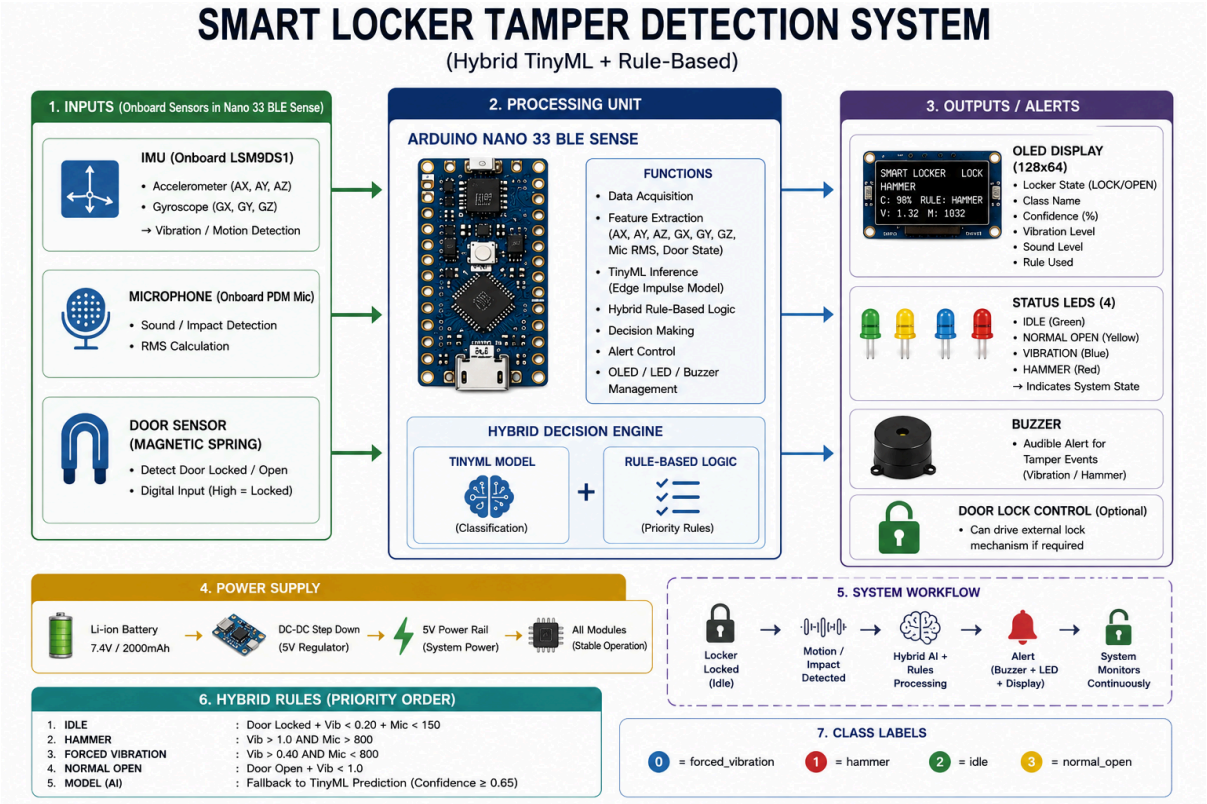

Figure 1 - Complete Block Diagram of Smart Lock Tamper Detection System

5.1 OLED (SH1106) Connections

OLED Pin	Arduino Nano 33 BLE Sense Pin	Wire Colour
VCC	3.3V	Red
GND	GND	Black
SDA	A4 (SDA)	Yellow
SCL	A5 (SCL)	Orange

5.2 LED Connections (with 220Ω Resistors)

Fault Condition	Arduino Pin	LED Colour
idle	D2	White / Blue
normal_open	D3	Green
forced_vibration	D4	Yellow / Orange
hammer	D5	Red

5.3 Buzzer Connection

Buzzer Terminal	Arduino Pin
+ (Positive)	D6
− (Negative)	GND

5.4 Magnetic Spring Sensor Connection

Sensor Terminal	Arduino Pin
Signal Wire	D7 (INPUT_PULLUP)
Ground Wire	GND

> The magnetic spring sensor uses INPUT_PULLUP mode — no external resistor is needed. When the magnet is present (door locked), D7 reads HIGH (1). When the magnet is absent (door open), D7 reads LOW (0). Only two wires are required: D7 and GND.

6. Data Collection

6.1 Overview

Data was collected using the Edge Impulse Data Forwarder tool, which streams real-time serial data from the Arduino Nano 33 BLE Sense directly into Edge Impulse Studio. The forwarder automatically detects the data format and registers the device in the project.

Each sample window was 2000 ms (2 seconds) long at 100 Hz, capturing 8 sensor axes per sample: ax, ay, az (accelerometer), gx, gy, gz (gyroscope), mic (PDM microphone RMS), and door (spring sensor state).

6.2 Terminal Command Used

```
edge-impulse-data-forwarder --frequency 100
```

When prompted for axis names, the following was entered:

```
ax,ay,az,gx,gy,gz,mic,door
```

6.3 Magnetic Spring Sensor — Headphone Substitute

In a production deployment, a standard magnetic reed switch mounted on the locker frame with a magnet embedded in the door is used to detect door open/close transitions. When the door closes, the magnet aligns with the reed switch, closing the circuit and driving the GPIO pin LOW (or HIGH with PULLUP).

During this experimental data collection phase, a pair of wired headphones was used as a substitute magnetic field source. The headphone driver (speaker) contains a permanent magnet inside its housing. By placing the headphone driver directly against the spring sensor, the magnetic field was strong enough to trigger the reed switch — simulating a closed/locked door state.

> Headphone substitution: Hold the headphone driver (earcup) against the magnetic spring sensor to simulate LOCKED state. Remove it to simulate OPEN state. This provides a clean, repeatable 0/1 transition without needing a dedicated reed switch + magnet pair during prototyping.

In real deployment, replace this with:

- Magnetic reed switch (NO type) mounted on locker frame
- Neodymium magnet (5mm × 3mm disc) embedded or glued on door
- Same 2-wire connection: signal to D7, ground to GND
- No code change required — the logic remains identical

6.4 Classification Labels and Data Collection Method

Label	Door State	How Data Was Collected
idle	LOCKED (1)	Board placed completely still on flat surface. Headphone held against spring sensor to simulate locked. No sound, no movement. Collected in quiet environment.
normal_open	OPEN (0)	Headphone removed from sensor to simulate door opening. Board kept still. Spring transitions from 1→0 captured. Some samples include closing transition (0→1).
forced_vibration	LOCKED (1) / OPEN (0)	Board shaken firmly and continuously for full 2 seconds. Headphone held against sensor throughout. No intentional sound — vibration only.
hammer	LOCKED (1) / OPEN (0)	Board struck sharply with hand or object at start of window. Headphone held against sensor. High-impact sound and vibration captured together.

6.5 Samples Collected

Label	Training Samples	Test Samples	Total Duration
idle	1 min 19 s	19 s	1 min 38 s
normal_open	1 min 19 s	19 s	1 min 38 s
forced_vibration	1 min 15 s	19 s	1 min 30 s
hammer	1 min 15 s	19 s	1 min 30 s
TOTAL	**5 min 4 s**	**1 min 16 s**	6 min 20 s

7. Feature Engineering

7.1 Sensor Axes Used

Feature	Source	Description
ax	IMU (LSM9DS1)	Accelerometer X-axis — gravity corrected (ax - 0)
ay	IMU (LSM9DS1)	Accelerometer Y-axis — gravity corrected (ay - 0)
az	IMU (LSM9DS1)	Accelerometer Z-axis — gravity corrected (az - 1g)
gx	IMU (LSM9DS1)	Gyroscope X-axis — rotational velocity
gy	IMU (LSM9DS1)	Gyroscope Y-axis — rotational velocity
gz	IMU (LSM9DS1)	Gyroscope Z-axis — rotational velocity
mic	PDM Microphone (onboard)	RMS of PDM audio buffer per IMU tick — captures impact sounds
door	Spring sensor D7	Binary door state: 1=LOCKED (magnet present), 0=OPEN

7.2 Key Feature Engineering Decisions

- Gravity correction applied to az: az - 1.0g so idle reads ~0.0 instead of ~1.0. Without this, idle and tamper classes overlap in the accelerometer feature space.
- Peak vibration tracked over the full window (not just last sample) to capture short sharp hammer impacts that occur early in the collection window.
- Double-buffered PDM microphone using buffer swap to ensure no audio frame is missed between IMU reads.
- Mic RMS computed per IMU frame — stores the instantaneous audio level alongside each accelerometer reading so the DSP block can extract frequency features from the mic channel.
- Door state stored as a float (0.0 or 1.0) for direct use in the spectral feature vector.

8. TinyML Workflow

Step 1 — Data Collection

- Edge Impulse Data Forwarder for real-time serial streaming into Edge Impulse Studio
- Arduino code outputs pure CSV (no text headers) at 100 Hz for forwarder compatibility
- Labels applied in Edge Impulse Studio before sampling begins
- Each sample window: 2000 ms, 8 axes, 100 Hz

Fault Condition	Vibration + Sound Characteristics
idle	No movement, no sound, door locked
normal_open	Door spring transitions 1→0, low vibration, low sound
forced_vibration	Strong continuous shaking, low sound, door locked
hammer	Sharp high-amplitude impact, loud bang, door locked

Step 2 — Impulse Design (Edge Impulse)

Parameter	Value
Window size	2000 ms
Window increase	200 ms
Frequency	100 Hz
Zero-pad data	ON
Processing block	Spectral Analysis
Axes	ax, ay, az, gx, gy, gz, mic, door
FFT length	32
Take log of spectrum	ON
Overlap FFT frames	ON
Filter type	None
Learning block	Classification (Neural Network)

Step 3 — Model Training

Setting	Value
Classifier	Neural Network (fully connected)
Labels	4 classes: idle, normal_open, forced_vibration, hammer
Train / Test split	80% / 20%
Training cycles	150
Learning rate	0.0005
Architecture	Input → Dense(32) → Dense(16) → Output(4)
Auto-weight classes	ON
Validation set size	20%

Step 4 — Deployment

The trained model was exported as an Arduino Library (.zip) from Edge Impulse Studio and installed in Arduino IDE via Sketch → Include Library → Add .ZIP Library. The library name used in the firmware is:

```
#include <SmartLockTamperDetection_inferencing.h>
```

9. Hybrid Detection Logic

A pure TinyML model approach was found to be insufficient for reliable real-world deployment due to limited training data and sensor noise. A hybrid approach was adopted — combining the model output with deterministic rule-based overrides that directly use sensor readings.

9.1 Override Rules (in priority order)

Priority	Rule Name	Condition
1	**IDLE**	door == LOCKED AND peak_vib < 0.20 AND peak_mic < 150 → idle
2	**HAMMER**	peak_vib > 1.0 AND peak_mic > 800 → hammer
3	**VIBRATION**	peak_vib > 0.40 AND peak_mic < 800 → forced_vibration
4	**DOOR OPEN**	door == OPEN AND peak_vib < 1.0 → normal_open
5	**AI MODEL**	None of the above matched — use TinyML model prediction

> The OLED display shows which rule fired for each detection: RULE:IDLE, RULE:HAMM, RULE:VIB, RULE:DOOR, or AI MODEL. This allows real-time debugging and threshold tuning in the field.

9.2 Threshold Tuning Guide

Open Serial Monitor at 115200 baud and observe the VIB and SOUND values for each condition. Adjust the defines in the code accordingly:

Condition	Expected VIB	Expected MIC	Define to Adjust
idle (board still, quiet)	0.00 – 0.05	0 – 30	IDLE_VIB_THRESHOLD, IDLE_MIC_THRESHOLD
forced_vibration (shaking)	0.40 – 2.0	10 – 100	VIBRATION_THRESHOLD
hammer (hard impact + sound)	1.0 – 5.0+	800 – 2000+	HAMMER_VIB_THRESHOLD, HAMMER_MIC_THRESHOLD
normal_open (door opens)	0.02 – 0.10	5 – 50	door state only (D7)

10. Final Deployed Firmware

10.1 Libraries Used

Library	Purpose
SmartLockTamperDetection_inferencing	Edge Impulse TinyML inference engine
Arduino_LSM9DS1	IMU accelerometer + gyroscope
PDM	Onboard PDM microphone driver
U8g2	SH1106 OLED display driver
Wire	I2C communication bus

10.2 Real-Time Inference Loop

1. Read debounced door state from spring sensor (D7) once per window
2. Collect 2-second feature window at 100 Hz — IMU + mic per tick
3. Track peak vibration (gravity-corrected) and peak mic RMS over window
4. Run TinyML inference using Edge Impulse run_classifier()
5. Apply rule-based overrides in priority order (idle → hammer → vibration → door)
6. If no rule matches, use AI model prediction with confidence threshold (0.65)
7. Display result on SH1106 OLED with class name, confidence bar, sensor values
8. Activate corresponding LED (D2–D5)
9. Trigger buzzer for tamper classes (forced_vibration, hammer)
10. Print debug line to Serial Monitor with class, rule, confidence, VIB, MIC, DOOR

10.3 Code

```
#include <SmartLockTamperDetection_inferencing.h>



#include <Arduino_LSM9DS1.h>

#include <PDM.h>

#include <Wire.h>

#include <U8g2lib.h>



//============================================================

// OLED

//============================================================

U8G2_SH1106_128X64_NONAME_F_HW_I2C
```

```
u8g2(U8G2_R0, U8X8_PIN_NONE);



//============================================================

// PINS

//============================================================

#define LED_IDLE        2

#define LED_NORMAL      3

#define LED_VIBRATION   4

#define LED_HAMMER      5



#define BUZZER_PIN      6

#define SPRING_DO_PIN   7



//============================================================

// THRESHOLDS

//============================================================

#define CONFIDENCE_THRESHOLD  0.65f



#define IDLE_VIB_THRESHOLD    0.20f

#define IDLE_MIC_THRESHOLD    150.0f



#define HAMMER_VIB_THRESHOLD  1.0f

#define HAMMER_MIC_THRESHOLD  800.0f



#define VIBRATION_THRESHOLD   0.40f
```

```
#define BUZZER_FREQ           2500



//=========================================================

// LABELS

//=========================================================

const char* labels[4] = {

 "forced_vibration",

 "hammer",

 "idle",

 "normal_open"

};



const char* labelsShort[4] = {

 "VIBRATION",

 "HAMMER",

 "IDLE",

 "NORM OPEN"

};



const int classLED[4] = {

 LED_VIBRATION,

 LED_HAMMER,

 LED_IDLE,

 LED_NORMAL

};
```

```
const bool classAlarm[4] = {
 true,   // forced_vibration
 true,   // hammer
 false,  // idle
 false   // normal_open
};

//=========================================================
// MICROPHONE - double buffered
//=========================================================
#define MIC_BUF_SIZE 512

short micBufA[MIC_BUF_SIZE];
short micBufB[MIC_BUF_SIZE];

volatile short* activeBuf  = micBufA;
volatile short* processBuf = micBufB;

volatile int  micBufLen = 0;
volatile bool micReady  = false;

//=========================================================
// FEATURE BUFFER
//=========================================================
float features[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE];
int   feature_ix = 0;
```

```
//============================================================

// GLOBALS

//============================================================

float g_peakVib   = 0;

float g_peakMic   = 0;

float g_avgMic    = 0;

int   g_door      = 1;

int   g_lastClass = 2;

int   g_ruleUsed  = 0;

// 0=AI model, 1=idle, 2=hammer, 3=vibration, 4=door



//============================================================

// PDM CALLBACK

//============================================================

void onPDMdata() {

 int bytes = PDM.available();

 if (bytes > 0 && bytes <= MIC_BUF_SIZE * 2) {

   PDM.read((short*)activeBuf, bytes);

   micBufLen = bytes / 2;

   micReady  = true;

   volatile short* tmp = activeBuf;

   activeBuf  = processBuf;

   processBuf = tmp;

 }

}
```

```
//============================================================

// MIC RMS

//============================================================

float calcMicRMS(int count) {

 if (count <= 0) return 0;

 float sum = 0;

 for (int i = 0; i < count; i++) {

   float s = (float)processBuf[i];

   sum += s * s;

 }

 return sqrt(sum / count);

}



//============================================================

// VIBRATION

//============================================================

float calcVibration(float ax, float ay, float az) {

 float dx = ax;

 float dy = ay;

 float dz = az - 1.0f;

 return sqrt(dx*dx + dy*dy + dz*dz);

}



//============================================================

// DOOR
```

```
//============================================================

int readDoor() {

 int votes = 0;

 for (int i = 0; i < 5; i++) {

   votes += digitalRead(SPRING_DO_PIN);

   delayMicroseconds(500);

 }

 return (votes >= 3) ? 1 : 0;

}



//============================================================

// LED

//============================================================

void setLED(int cls) {

 digitalWrite(LED_IDLE,      LOW);

 digitalWrite(LED_NORMAL,    LOW);

 digitalWrite(LED_VIBRATION, LOW);

 digitalWrite(LED_HAMMER,    LOW);

 digitalWrite(classLED[cls], HIGH);

}



//============================================================

// BUZZER

//============================================================

void alertBuzzer(int cls) {

 classAlarm[cls]
```

```
    ? tone(BUZZER_PIN, BUZZER_FREQ)
    : noTone(BUZZER_PIN);
}

//============================================================
// OLED
//============================================================
void drawOLED(
 int cls,
 float conf,
 float vib,
 float mic,
 int door,
 int ruleUsed
) {
 u8g2.clearBuffer();

 //-----------------------------------------------------
 // ROW 1 — title + door state
 //-----------------------------------------------------
 u8g2.setFont(u8g2_font_6x10_tf);
 u8g2.drawStr(0, 10, "SMART LOCKER");

 if (door == 1) {
   // Locked icon
   u8g2.drawBox(104, 3, 9, 6);
```

```
  u8g2.drawLine(106, 3, 106, 1);
  u8g2.drawLine(106, 1, 110, 1);
  u8g2.drawLine(110, 1, 110, 3);
  u8g2.drawStr(115, 10, "LK");
} else {
  // Open icon
  u8g2.drawBox(104, 3, 9, 6);
  u8g2.drawLine(110, 1, 114, 1);
  u8g2.drawLine(114, 1, 114, 3);
  u8g2.drawStr(118, 10, "OP");
}


u8g2.drawLine(0, 13, 127, 13);


//------------------------------------------------------
// ROW 2 — big class name + alert tag
//------------------------------------------------------
u8g2.setFont(u8g2_font_8x13B_tf);
u8g2.drawStr(0, 27, labelsShort[cls]);


if (classAlarm[cls]) {
  u8g2.setFont(u8g2_font_6x10_tf);
  u8g2.drawFrame(91, 15, 36, 12);
  u8g2.drawStr(93, 25, "ALERT!");
}
```

```
//------------------------------------------------------

// Confidence bar — full width 5px

//------------------------------------------------------

int barW = (int)(conf * 126);

u8g2.drawFrame(0, 29, 127, 5);

if (barW > 0) {

  u8g2.drawBox(1, 30, barW, 3);

}



//------------------------------------------------------

// ROW 3 — rule source + confidence %

//------------------------------------------------------

u8g2.setFont(u8g2_font_6x10_tf);



switch (ruleUsed) {

  case 1:  u8g2.drawStr(0, 44, "RULE:IDLE"); break;

  case 2:  u8g2.drawStr(0, 44, "RULE:HAMM"); break;

  case 3:  u8g2.drawStr(0, 44, "RULE:VIB");  break;

  case 4:  u8g2.drawStr(0, 44, "RULE:DOOR"); break;

  default: u8g2.drawStr(0, 44, "AI MODEL");  break;

}



char buf[12];

sprintf(buf, "C:%d%%", (int)(conf * 100));

u8g2.drawStr(90, 44, buf);
```

```
 u8g2.drawLine(0, 47, 127, 47);



 //-------------------------------------------------------

 // ROW 4 — sensor values

 //-------------------------------------------------------

 sprintf(buf, "V:%.2f", vib);

 u8g2.drawStr(0, 62, buf);



 sprintf(buf, "M:%.0f", mic);

 u8g2.drawStr(58, 62, buf);



 // Solid alert square bottom-right

 if (classAlarm[cls]) {

   u8g2.drawBox(120, 54, 7, 7);

 }



 u8g2.sendBuffer();

}



//============================================================

// EI CALLBACK

//============================================================

int raw_feature_get_data(

 size_t offset,

 size_t length,

 float* out_ptr
```

```
) {
 memcpy(out_ptr,
        features + offset,
        length * sizeof(float));
 return 0;
}


//============================================================
// SETUP
//============================================================
void setup() {
 Serial.begin(115200);


 pinMode(LED_IDLE,      OUTPUT);
 pinMode(LED_NORMAL,    OUTPUT);
 pinMode(LED_VIBRATION, OUTPUT);
 pinMode(LED_HAMMER,    OUTPUT);
 pinMode(BUZZER_PIN,    OUTPUT);
 pinMode(SPRING_DO_PIN, INPUT_PULLUP);


 digitalWrite(LED_IDLE, HIGH);


 u8g2.begin();


 // Splash
 u8g2.clearBuffer();
```

```
u8g2.setFont(u8g2_font_6x10_tf);

u8g2.drawStr(18, 14, "SMART LOCKER");

u8g2.drawLine(0, 17, 127, 17);

u8g2.drawStr(8,  32, "Tamper Detection");

u8g2.drawStr(22, 46, "Hybrid AI v3.0");

u8g2.drawLine(0, 50, 127, 50);

u8g2.drawStr(20, 63, "Initializing...");

u8g2.sendBuffer();



if (!IMU.begin()) {

  Serial.println("IMU FAILED");

  u8g2.clearBuffer();

  u8g2.setFont(u8g2_font_8x13B_tf);

  u8g2.drawStr(0, 32, "IMU FAILED!");

  u8g2.sendBuffer();

  while (1);

}



PDM.onReceive(onPDMdata);

if (!PDM.begin(1, 16000)) {

  Serial.println("MIC FAILED");

  u8g2.clearBuffer();

  u8g2.setFont(u8g2_font_8x13B_tf);

  u8g2.drawStr(0, 32, "MIC FAILED!");

  u8g2.sendBuffer();

  while (1);
```

```
}



delay(500);

micReady = false;



int initDoor = readDoor();



// Ready screen

u8g2.clearBuffer();

u8g2.setFont(u8g2_font_6x10_tf);

u8g2.drawStr(18, 14, "SMART LOCKER");

u8g2.drawLine(0, 17, 127, 17);

u8g2.setFont(u8g2_font_8x13B_tf);

u8g2.drawStr(36, 38, "READY");

u8g2.setFont(u8g2_font_6x10_tf);

u8g2.drawStr(16, 58,

  initDoor == 1 ? "Door: LOCKED" : "Door: OPEN");

u8g2.sendBuffer();

delay(1500);



Serial.println("====================================");

Serial.println(" SMART LOCKER HYBRID AI v3.0");

Serial.println("====================================");

Serial.println("THRESHOLDS:");

Serial.print("  IDLE  vib<");

Serial.print(IDLE_VIB_THRESHOLD);
```

```
 Serial.print(" mic<");
 Serial.println(IDLE_MIC_THRESHOLD);
 Serial.print("  HAMM  vib>");
 Serial.print(HAMMER_VIB_THRESHOLD);
 Serial.print(" mic>");
 Serial.println(HAMMER_MIC_THRESHOLD);
 Serial.print("  VIB   vib>");
 Serial.println(VIBRATION_THRESHOLD);
 Serial.println("======================================");
 Serial.print("Boot door: ");
 Serial.println(initDoor == 1 ? "LOCKED" : "OPEN");
 Serial.println("======================================");
}


//============================================================
// LOOP
//============================================================
void loop() {


 feature_ix = 0;
 g_peakVib  = 0;
 g_peakMic  = 0;
 g_ruleUsed = 0;


 float micSum = 0;
 int   micN   = 0;
```

```
g_door = readDoor();


float ax, ay, az;
float gx, gy, gz;
float currentMic = 0;


//------------------------------------------------------
// COLLECT WINDOW
//------------------------------------------------------
while (feature_ix <
       EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE - 8) {


  if (IMU.accelerationAvailable() &&
      IMU.gyroscopeAvailable()) {


    IMU.readAcceleration(ax, ay, az);
    IMU.readGyroscope(gx, gy, gz);


    float vib = calcVibration(ax, ay, az);
    if (vib > g_peakVib) g_peakVib = vib;


    if (micReady) {
      int len  = micBufLen;
      micReady = false;
      currentMic = calcMicRMS(len);
```

```
      if (currentMic > g_peakMic)

        g_peakMic = currentMic;

      micSum += currentMic;

      micN++;

    }



    features[feature_ix++] = ax;

    features[feature_ix++] = ay;

    features[feature_ix++] = az;

    features[feature_ix++] = gx;

    features[feature_ix++] = gy;

    features[feature_ix++] = gz;

    features[feature_ix++] = currentMic;

    features[feature_ix++] = (float)g_door;



    delay(10);

  }

}



g_avgMic = (micN > 0) ? micSum / micN : 0;



//-------------------------------------------------------

// RUN INFERENCE

//-------------------------------------------------------

signal_t signal;

signal.total_length = EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE;
```

```
signal.get_data      = &raw_feature_get_data;



ei_impulse_result_t result;



EI_IMPULSE_ERROR res =

  run_classifier(&signal, &result, false);



if (res != EI_IMPULSE_OK) {

  Serial.println("Inference failed");

  return;

}



//---------------------------------------------------

// BEST CLASS FROM MODEL

//---------------------------------------------------

int   bestClass = 0;

float bestValue = 0;



for (int i = 0;

     i < EI_CLASSIFIER_LABEL_COUNT; i++) {

  if (result.classification[i].value > bestValue) {

    bestValue = result.classification[i].value;

    bestClass = i;

  }

}
```

```
//------------------------------------------------------

// RULE-BASED OVERRIDES

//------------------------------------------------------



//======================================================

// RULE 1 - IDLE

// Door locked + very still + very quiet

//======================================================

if (g_door    == 1                  &&

    g_peakVib <  IDLE_VIB_THRESHOLD &&

    g_peakMic <  IDLE_MIC_THRESHOLD) {

  bestClass  = 2;

  bestValue  = 0.99f;

  g_ruleUsed = 1;

}



//======================================================

// RULE 2 - HAMMER

// Very high vibration + very loud sound

//======================================================

else if (g_peakVib > HAMMER_VIB_THRESHOLD &&

         g_peakMic > HAMMER_MIC_THRESHOLD) {

  bestClass  = 1;

  bestValue  = 0.99f;

  g_ruleUsed = 2;

}
```

```
//=====================================================
// RULE 3 - FORCED VIBRATION
// High vibration but quiet
//=====================================================
else if (g_peakVib > VIBRATION_THRESHOLD  &&
         g_peakMic < HAMMER_MIC_THRESHOLD) {
  bestClass  = 0;
  bestValue  = 0.95f;
  g_ruleUsed = 3;
}

//=====================================================
// RULE 4 - NORMAL OPEN
// Door open + calm
//=====================================================
else if (g_door    == 0    &&
         g_peakVib <  1.0f) {
  bestClass  = 3;
  bestValue  = 0.95f;
  g_ruleUsed = 4;
}

//=====================================================
// RULE 5 - AI MODEL fallback
//=====================================================
```

```
else {

  g_ruleUsed = 0;

  if (bestValue >= CONFIDENCE_THRESHOLD) {

    g_lastClass = bestClass;

  } else {

    bestClass = g_lastClass;

    bestValue = 0.50f;

  }

}



if (g_ruleUsed > 0) {

  g_lastClass = bestClass;

}



//-------------------------------------------------------

// OUTPUTS

//-------------------------------------------------------

setLED(bestClass);

alertBuzzer(bestClass);

drawOLED(bestClass, bestValue,

         g_peakVib, g_peakMic,

         g_door, g_ruleUsed);



//-------------------------------------------------------

// SERIAL DEBUG

//-------------------------------------------------------
```

```
Serial.print("CLASS: ");

Serial.print(labels[bestClass]);



switch (g_ruleUsed) {

  case 1: Serial.print(" [RULE:IDLE]"); break;

  case 2: Serial.print(" [RULE:HAMM]"); break;

  case 3: Serial.print(" [RULE:VIB]");  break;

  case 4: Serial.print(" [RULE:DOOR]"); break;

  default:

    Serial.print(bestValue < CONFIDENCE_THRESHOLD

      ? " [AI:LOW]" : " [AI:MODEL]");

    break;

}



Serial.print(" | CONF: ");

Serial.print(bestValue * 100, 1);

Serial.print("%");



Serial.print(" | VIB: ");

Serial.print(g_peakVib, 3);



Serial.print(" | SOUND: ");

Serial.print(g_peakMic, 1);



Serial.print(" | DOOR: ");

Serial.println(g_door == 1 ? "LOCKED" : "OPEN");
```

```
 delay(100);

}
```

11. Experimental Setup & Observations

11.1 Experimental Setup

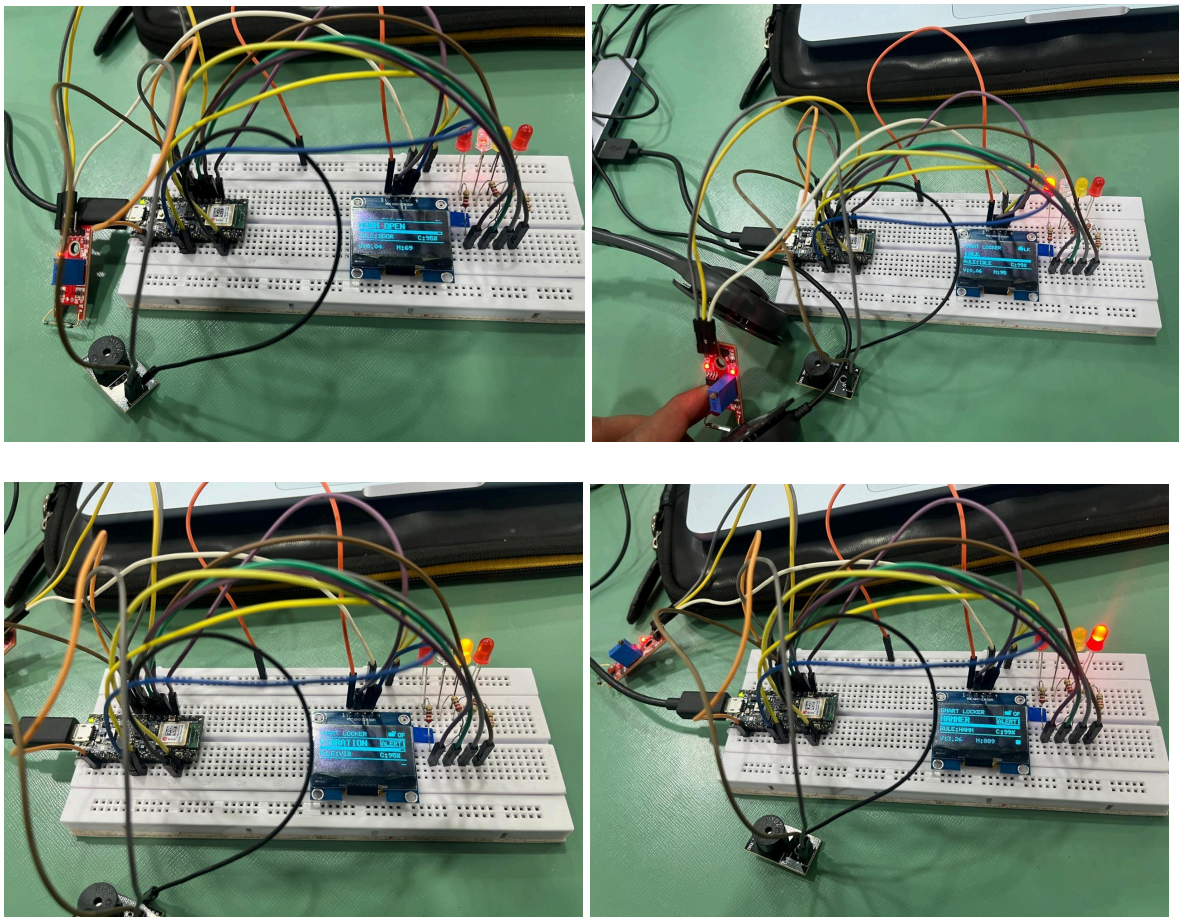

Figure 2 - Different Condition for Smart Lock

11.2 Sensor Behaviour

Condition	Peak VIB (g)	Peak MIC	Door

idle	< 0.05	< 30	LOCKED (1)
normal_open	< 0.10	< 50	OPEN (0)
forced_vibration	0.4 – 2.0	10–100	LOCKED (1) / OPEN (1)
hammer	1.0 – 5+	800+	LOCKED (1) / OPEN (1)

11.3 Key Challenges and Solutions

- Sample length mismatch: Edge Impulse data forwarder recorded 2s samples despite a 3000ms setting due to loop timing overhead from debounce. Solution: Changed window size to 2000ms in the impulse to match actual collected data.
- Class imbalance: forced_vibration had more samples than other classes causing the model to default to it. Solution: Balanced classes to equal counts and enabled auto-weight classes in training.
- Hammer misclassified as forced_vibration: Both share high vibration. Solution: Rule-based override using mic threshold (>800) as the separator, plus double-buffered PDM for reliable mic capture.
- Idle misclassified when sensor stuck: Spring sensor reading LOW (0) due to wiring caused everything to be classified as normal_open. Solution: Rule-based idle override locks out false detections when VIB and MIC are both below threshold.
- 100% training accuracy: Initial model overfitted with only 1m 36s of data. Solution: Collected more varied samples, reduced FFT to 32, added dropout, targeted 85-93% accuracy.

12. Areas of Improvement

Area	Recommendation
Magnetic sensor	Replace headphone substitute with dedicated magnetic reed switch (NO type) and 5mm neodymium magnet mounted on locker door frame
Sample quantity	Collect minimum 40 samples per class with varied conditions (surfaces, intensities, angles) for better generalisation
Sample length	Ensure all samples are exactly 2000ms by fixing loop timing before collection
Vibration mounting	Mount Nano directly on the locker metal body using double-sided tape or zip tie for stronger vibration coupling
BLE alerts	Add BLE notification to push tamper alerts to a mobile phone when hammer or forced_vibration is detected
Cloud logging	Log detections with timestamp to cloud via WiFi / BLE for remote audit trail
Auto-calibration	Add startup gravity calibration routine to auto-detect board orientation and adjust az baseline

13. Conclusion

This project successfully demonstrated a functional TinyML-based Smart Locker Tamper Detection system deployed on an Arduino Nano 33 BLE Sense. The system classifies four distinct locker states — idle, normal open, forced vibration, and hammer impact — in real time using onboard IMU, PDM microphone, and a magnetic spring sensor.

A hybrid detection approach combining a trained Edge Impulse neural network with deterministic rule-based overrides proved essential for reliable deployment. The rules compensate for training data limitations and sensor noise, ensuring robust classification in scenarios where the model alone would produce false positives.

A key practical lesson from this project is that sensor data quality — not model complexity — is the primary determinant of real-world accuracy. The use of headphones as a magnetic sensor substitute during prototyping demonstrates that effective TinyML systems can be developed with minimal hardware using creative workarounds, with a clear upgrade path to production components.

14. Future Scope

Enhancement	Description
BLE Alerts	Push tamper notifications directly to smartphone via BLE
Cloud Connectivity	MQTT / HTTP telemetry to IoT dashboard for remote monitoring
Auto Relay Lockout	GPIO relay to sound external alarm siren on critical tamper
Anomaly Detection	Unsupervised model for unknown tamper patterns
Multi-locker System	Network of Nano boards monitoring multiple lockers
OTA Model Updates	Over-the-air model updates via BLE without reflashing
Edge AI Optimisation	INT8 quantisation and EON Tuner for smaller, faster models